Clustering vs Classification and the K-Means Algorithm

Basics of Data Mining — Week 7

Index

- 1. Where We Are in the Data Mining Journey — *We have the data, we have trimmed the data, now we finally get to do the "mining" in data mining.*
- 2. The One Question That Decides Everything — *"Do I have labels?" — the only question that matters, and it has only two answers.*
- 3. Classification with Images — *When a million images come with name tags.*
- 4. Clustering with Images — *When the million images show up without name tags and you must make friends anyway.*
- 5. Why Clustering Needs Post-Analysis — *The algorithm hands you piles; you explain why the piles exist.*
- 6. Domain Knowledge: Loan Default Prediction — *A toy bank, three customers, and one very classic classification problem.*
- 7. Discovering Customer Segments — *Same bank, no labels, now we play marketing detective.*
- 8. Client Says "Maximize Profit" — Now What? — *Nobody hands you a problem labelled "classification". You decide.*
- 9. K-Means Clustering: The Idea — *K groups, K averages, zero mystery.*
- 10. K-Means Setup and Notation — *Meet x_i, d, N and K.*
- 11. K-Means Procedure Step by Step — *Guess, assign, average, repeat until bored (formally: until the centroids stop moving).*
- 12. A Fully Worked Numerical Example — *Real arithmetic, six points, two clusters, no skipping.*
- 13. What Is K-Means Actually Doing? — *The hidden objective function behind the procedure.*
- 14. Summary of the Lecture — *The one-minute version.*
- 15. Practice Questions — *Five questions that make you think, not recite.*

---

1. Where We Are in the Data Mining Journey

So far the course has covered the background that anyone doing data mining needs:

- certain aspects of databases and data mining, and the common terms
- data reduction
- features, attributes and characteristics

From here we move to **processing of the data**. The data is available, and some data reduction may already have been done. Now comes the actual **mining**.

One of the main things we can do in mining is **supervised** and **unsupervised** learning. You may know these terms from a machine learning course. This lecture introduces two ideas that sit under them:

Idea	Learning Type
Clustering	Unsupervised learning
Classification	Supervised learning

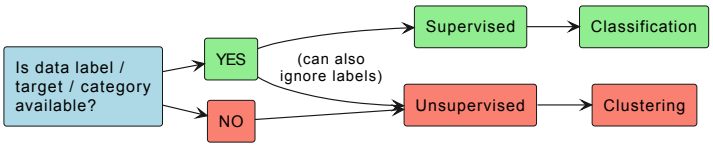
The main focus of this lecture, and of the course in general, is **clustering**.

2. The One Question That Decides Everything

To decide between clustering and classification, there is only one question to ask:

> **Is the data label (target / category) available?**

- If the label is known, the problem can be **supervised**, which is **classification**.
- If the label is not known, all we can do is **unsupervised** learning, which is **clustering**.



Key observations

- If the answer is **yes**, you can do **both** classification and clustering. You may simply ignore the labels, or use them later for verification.
- If the answer is **no**, **clustering** is the only option.
- So clustering is the more *general* idea, since you can do more things with it.
- Classification is fixed: you know what to do, you know your target, and you know where you must end up. Clustering is the slightly more interesting and slightly more challenging one.

> **Note for Review** Think of **classification** as a quiz with an answer key (you are graded against known labels), and **clustering** as sorting a pile of unlabelled photographs into albums on your own. In the second case, nobody can tell you the "correct" albums. You can only judge whether your albums are *useful*.

3. Classification with Images

Suppose we have **10 lakh (1 million) images**, and each image comes with a **label** that says which animal it contains.

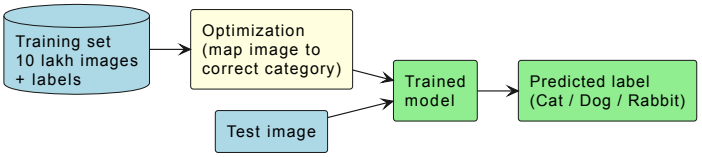
Some facts about the scale of this data:

- A single full-HD image has 1920×1080 pixels.
- That is $1920 \times 1080 = 2{,}073{,}600$ pixels for **one** image.
- With 10^6 images, the data set holds roughly 2.07×10^{12} pixel values. That is a lot of data.

Since the **class label is known**, we do the optimization (training of the network) **towards the correct category**. This is **classification**.

Image	Label
1	Cat
2	Dog
3	Dog
4	Rabbit
5	Cat
...	...

Testing: when a new *test image* is given, the trained network must tell us which category it belongs to (cat, dog, rabbit, and so on).



4. Clustering with Images

Now keep the same 10^6 images, but **ignore the labels** (or assume they were never given).

This is a more challenging problem. We have a data set

$$D = \{x_i\}, \quad i = 1, 2, \ldots, 10^6$$

where each x_i is an image (a vector or matrix of pixels). The central question is:

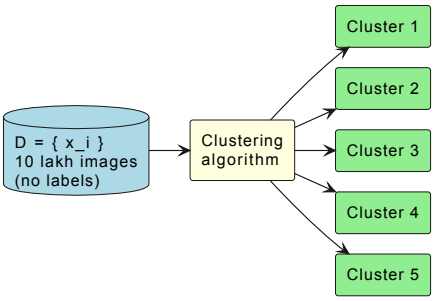
> **Is there a useful or interesting pattern in D?**

We do not know anything about the labels. We are simply asking whether the data tells us something on its own. This question belongs to **clustering**.

Suppose the analysis says there are **5 clusters** (5 is just an example):

- Some images fall into cluster 1, others into cluster 2, and so on.
- Images in the same cluster share something in common, a pattern that is useful for that cluster.
- The next cluster has some *other* common information that put those images together.

Notice that the algorithm tells us **nothing** about the labels. It only says "these images form 5 groups".



5. Why Clustering Needs Post-Analysis

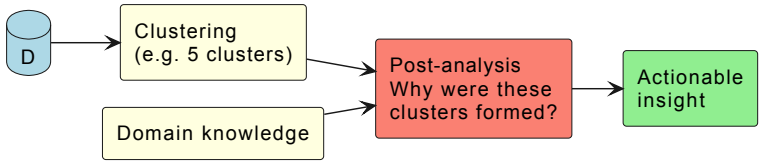
The interesting part is what happens **after** the clustering. Two questions remain that the algorithm does not answer:

1. **Was the clustering correct?** For instance, an image that should have gone to cluster 5 might have landed in cluster 1. We have to worry about this.
2. **Why were these clusters formed at all?** Even if the clustering is perfect, we must understand what each cluster *means* in the context of the application. This needs a **secondary analysis**.

Back to the animal example. The clustering algorithm may operate only at the **pixel level**, so it does not "know" what a cat is. It is unlikely that cats go to one cluster, dogs to another and rabbits to a third. It may instead put cats and dogs together (because of some shared visual commonality) and keep rabbits separate. Our job is to work out *why*.

Application	What the clusters might reveal
Patient records (10^6 patients)	Blood pressure and heart attack may be correlated, so one cluster holds patients with a **higher risk of heart attack**.
Credit card transactions	A cluster of unusual behaviour could point to **fraud**.
Medical imaging	Clusters may correspond to different **tumour** types.

So there is still an element of surprise after clustering. That is what makes it interesting.



The role of the data engineer. Almost anybody can run a clustering algorithm, because it is just a piece of code: give it data, run it, get output. The harder and more valuable part is explaining *why* you got that output. A logical, useful explanation (for example, in credit card fraud detection) is a very useful input for implementation and has a large impact.

> Running the algorithm and walking away is **not** the goal. You must give inputs based on your clustered (or even classified) data.

> **Note for Review Pro-Tip:** Exam answers on clustering are rewarded for **interpretation**, not just procedure. Remember that the algorithm only outputs *group memberships*. Giving the groups meaning (such as "high-risk patients" or "frugal investors") is a human, domain-driven step.

6. Domain Knowledge: Loan Default Prediction

Domain knowledge (medical imaging, banking, autonomous vehicles and so on) is essential. Without it you cannot decide which scenarios matter. Here is a toy example from banking.

Loan default prediction. A bank keeps the history of its customers: income, debt (how much they owe to other people) and the final outcome of the loan.

Customer	Income	Debt	Loan Result
A	40k	Low	Repaid
B	35k	High	Defaulted
C	80k	Low	Repaid

The question: *Will a given (new) customer default?* We have no idea in advance, but we can use this earlier data.

Since we **have the labels** (the final result column), this is a **classification** problem with only two possible classes:

$$\text{Class} \in \{\text{Default},\ \text{Repay}\}$$

How *successful* the prediction will be is another story. As far as the **problem setting** goes, it is classification.

7. Discovering Customer Segments

Now change the problem. Instead of asking whether a person will default or repay, we want to **discover customer segments**, meaning to group customers into some "classes" (not hard-bound classes, but **clusters**).

Why do banks want this?

- It gives the overall profile of the customer base.
- When a new customer arrives, the bank has an idea how to pitch products, where the customer will eventually go, and how to maximize profit from that customer.
- Services can be offered in a more *targeted* manner.

Banks can see your spending through your history, so we use these (hypothetical) features:

Customer	Spending	Savings	Investment
A	High	Low	Low
B	High	Low	Low
C	Low	High	High
D	Low	High	High

Two natural segments appear:

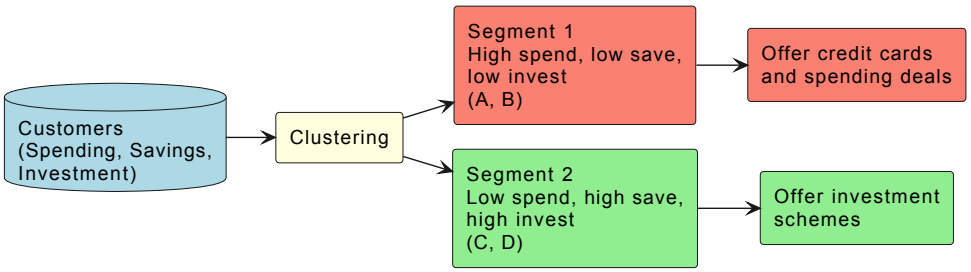
Segment	Spending	Savings	Investment	Customers
Segment 1	High	Low	Low	A, B
Segment 2	Low	High	High	C, D

This example is almost trivial (you can see the pattern by eye), but the idea scales to a big data set with many more columns.

Business use:

- High savers: pitch investment schemes.
- High spenders: pitch credit cards and offers.
- The bank diverts its effort to the *right* customer. The likelihood that a customer invests is identified by such patterns.

This is **clustering**, because we have **no label** and we are not pre-deciding that there are exactly these categories. We are *exploring*. Often it is not very high-level data mining: simple observations give real insight.



8. Client Says "Maximize Profit" — Now What?

In practice, clients rarely say "do classification" or "do clustering". They say something like:

> *"Here is our data D. Maximize our profit in terms of product sales or services."*

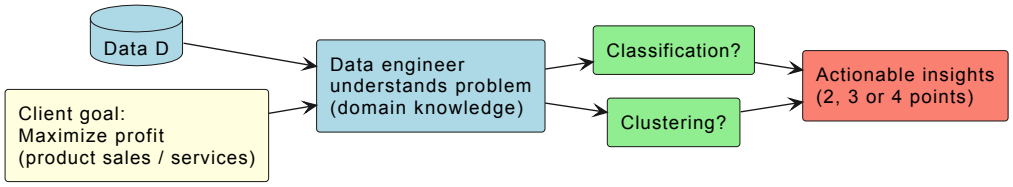
This is an abstract, broad statement. **As the data engineer, you decide** whether classification or clustering suits it better.

- **Classification route:** find labels if they exist. If they do not, you can try to *generate* them (for example, define what the final goal is for these people).
- **Clustering route:** run a clustering algorithm and see what structure appears.
- **Both together** is also possible.

What truly matters is not the label "classification" or "clustering". It is that **you understand the problem** the client is asking about. For that you need to:

1. understand the history of the problem,
2. look at the data,
3. ask for more data or information where needed (the process involves some back and forth).

There is no "right" or "wrong" between classification and clustering. The difference is whether your approach gives a **better insight** or only a general one. That depends on the effort you put into understanding the problem.



Actionable insight. After clustering, you should not claim a "perfect" statement of what the client must do, because nothing is perfect. You say: *"These three or four points have come out of my analysis."* The client then thinks it over with their own biases and constraints. As a data engineer, you provide the actionable insights.

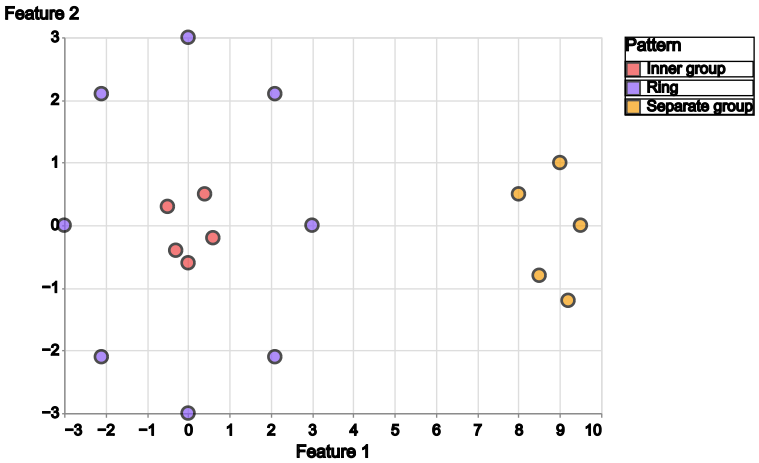
> **Note for Review Pro-Tip:** Questions often ask you to choose between clustering and classification for a described scenario. The deciding factor is the **availability and usefulness of a target label**, plus what the business actually wants. Do not just recite the definitions.

---

9. K-Means Clustering: The Idea

Assume we have decided to do clustering and have already brought in our domain knowledge. We now study a very basic algorithm: **K-Means clustering**.

- **K** is the number of clusters. For example, $K = 3$ means three clusters.
- **Means** refers to the **average** (mean). Based on averaging, we form K clusters.



> It is not enough to say "I will use K-Means". You need a **physical intuition** of what the algorithm is doing, just as with DFT and PCA in data reduction. **Intuitive picture.** Imagine 2D data in which some points form a ring, some points sit in the middle of that ring, and a third bunch lies to the side. We want to separate these groups automatically. In reality data lives in many dimensions, so we cannot visualise it, and that is why we need an algorithm.

---

10. K-Means Setup and Notation

Let the data be

$$x_i \in \mathrm{R}^d, \quad i = 1, 2, \ldots, N$$

where

- x_i is one data point (it can be an image, a feature vector and so on),
- N is the number of data points,
- d is the number of features of each data point.

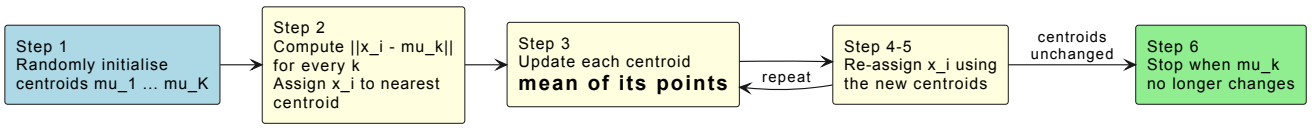
Running example, $d = 2$**:** the two features are *monthly spending* and *monthly savings*. So each data point is

$$x_i = \begin{pmatrix} \text{Spending} \\ \text{Savings} \end{pmatrix}$$

By convention the **first** number is spending and the **second** is savings.

Before starting, we **choose the number of clusters** K (2, 3, 4 and so on). All the steps below depend on this choice.

11. K-Means Procedure Step by Step



Step 1 — Randomly assign values to the centroids

Randomly choose values for

$$\mu_1, \mu_2, \ldots, \mu_K$$

Here μ_k is the **centroid** (the "centre" around which the data points of cluster k line up) of cluster k. We start with random values, as is typical in optimization (similar in spirit to starting gradient descent from a random point), and then narrow them down.

Since $d = 2$, **each centroid has two numbers** (a spending coordinate and a savings coordinate). For $K = 3$ the lecture used these arbitrary starting values:

$$\mu_1 = \begin{pmatrix} 0.9 \\ 1.9 \end{pmatrix}, \qquad \mu_2 = \begin{pmatrix} 7.2 \\ 2.7 \end{pmatrix}, \qquad \mu_3 = \begin{pmatrix} 1.7 \\ 7.4 \end{pmatrix}$$

(The numbers are assumed to be normalised.) Each centroid is a *point in the 2D spending and savings plane*, and the data points are spread across that same plane.

Step 2 — Evaluate Euclidean distances and assign

For every data point x_i, compute the Euclidean distance to each centroid:

$$\|x_i - \mu_k\| = \sqrt{(x_{i1} - \mu_{k1})^2 + (x_{i2} - \mu_{k2})^2}$$

With $K = 3$ this gives **three distances** per point. Then:

> **Assign x_i to the nearest centroid**, meaning the cluster with the smallest distance.

Written compactly, the cluster of point x_i is

$$c_i = \arg\min_k \|x_i - \mu_k\|$$

Small illustration with the centroids above.

Point $x = (2,\ 3)$*:*

$$d_1 = \sqrt{(2-0.9)^2 + (3-1.9)^2} = \sqrt{1.21 + 1.21} = \sqrt{2.42} \approx 1.556$$
$$d_2 = \sqrt{(2-7.2)^2 + (3-2.7)^2} = \sqrt{27.04 + 0.09} = \sqrt{27.13} \approx 5.209$$
$$d_3 = \sqrt{(2-1.7)^2 + (3-7.4)^2} = \sqrt{0.09 + 19.36} = \sqrt{19.45} \approx 4.410$$

The smallest is d_1, so this point joins **cluster 1**.

Point $x = (2,\ 6)$*:*

$$d_1 = \sqrt{(2-0.9)^2 + (6-1.9)^2} = \sqrt{1.21 + 16.81} = \sqrt{18.02} \approx 4.245$$
$$d_2 = \sqrt{(2-7.2)^2 + (6-2.7)^2} = \sqrt{27.04 + 10.89} = \sqrt{37.93} \approx 6.159$$
$$d_3 = \sqrt{(2-1.7)^2 + (6-7.4)^2} = \sqrt{0.09 + 1.96} = \sqrt{2.05} \approx 1.432$$

The smallest is d_3, so this point joins **cluster 3**.

Do this for every point x_i, $i = 1, \ldots, N$. After this pass every point has a cluster membership, and we have three populated clusters C_1, C_2, C_3.

Step 3 — Update the centroids

The random centroids were only a starting guess. Now replace each centroid with the **mean of all the points currently assigned to that cluster**:

$$\mu_k = \frac{1}{n_k} \sum_{x_i \in C_k} x_i$$

where

- C_k is the set of points in cluster k,
- n_k is the **number of points** in C_k.

Because x_i is 2D, the mean is also 2D, with an (average spending, average savings) pair:

$$\mu_k = \begin{pmatrix} \overline{\text{Spending}} \\ \overline{\text{Savings}} \end{pmatrix}$$

For instance, after the first update the lecture's example produced centroids such as

$$\mu_1 = \begin{pmatrix} 7.9 \\ 2.4 \end{pmatrix}, \qquad \mu_2 = \begin{pmatrix} 10.9 \\ 4.7 \end{pmatrix}, \qquad \mu_3 = \begin{pmatrix} 7.4 \\ 1.9 \end{pmatrix}$$

(These are illustrative numbers showing that the centroids *move* to new positions once they are replaced by cluster averages.)

Steps 4 and 5 — Re-assign using the new centroids

Using the **updated** centroids, compute $\|x_i - \mu_k\|$ again for every point and re-assign each x_i to its nearest new centroid.

Now some points may switch clusters. A point that was in cluster 1 may move to cluster 2, and so on. After re-assigning, **update the centroids again** (Step 3), then re-assign again (Step 5), and keep alternating.

Slowly but steadily, points settle into the nearest cluster, and the cluster centres settle too. The result becomes more and more meaningful.

Step 6 — Stop

> **Stop when μ_k is no longer changing.**

After a while the same points keep landing in the same clusters, so the new mean equals the previous mean. That is the stopping criterion.

---

12. A Fully Worked Numerical Example

To make every step concrete, here is a small example with $K = 2$ and $d = 2$ (spending, savings). The data points are:

Point	Spending	Savings
P_1	1	1
P_2	2	1
P_3	1	2
P_4	8	8
P_5	9	8
P_6	8	9

Step 1: initial (random) centroids

$$\mu_1 = \begin{pmatrix}2\\2\end{pmatrix}, \qquad \mu_2 = \begin{pmatrix}7\\7\end{pmatrix}$$

Step 2: distances (iteration 1)

$$\begin{aligned}
P_1 &: d_1 = \sqrt{(1-2)^2+(1-2)^2} = \sqrt{2} \approx 1.414, \quad d_2 = \sqrt{(1-7)^2+(1-7)^2} = \sqrt{72} \approx 8.485 \;\Rightarrow\; C_1\\
P_2 &: d_1 = \sqrt{(2-2)^2+(1-2)^2} = 1, \quad d_2 = \sqrt{(2-7)^2+(1-7)^2} = \sqrt{61} \approx 7.810 \;\Rightarrow\; C_1\\
P_3 &: d_1 = \sqrt{(1-2)^2+(2-2)^2} = 1, \quad d_2 = \sqrt{(1-7)^2+(2-7)^2} = \sqrt{61} \approx 7.810 \;\Rightarrow\; C_1\\
P_4 &: d_1 = \sqrt{(8-2)^2+(8-2)^2} = \sqrt{72} \approx 8.485, \quad d_2 = \sqrt{(8-7)^2+(8-7)^2} = \sqrt{2} \approx 1.414 \;\Rightarrow\; C_2\\
P_5 &: d_1 = \sqrt{(9-2)^2+(8-2)^2} = \sqrt{85} \approx 9.220, \quad d_2 = \sqrt{(9-7)^2+(8-7)^2} = \sqrt{5} \approx 2.236 \;\Rightarrow\; C_2\\
P_6 &: d_1 = \sqrt{(8-2)^2+(9-2)^2} = \sqrt{85} \approx 9.220, \quad d_2 = \sqrt{(8-7)^2+(9-7)^2} = \sqrt{5} \approx 2.236 \;\Rightarrow\; C_2
\end{aligned}$$

So $C_1 = \{P_1, P_2, P_3\}$ and $C_2 = \{P_4, P_5, P_6\}$.

Step 3: update the centroids (mean of each cluster)

$$\mu_1 = \frac{1}{3}\left[\begin{pmatrix}1\\1\end{pmatrix} + \begin{pmatrix}2\\1\end{pmatrix} + \begin{pmatrix}1\\2\end{pmatrix}\right] = \frac{1}{3}\begin{pmatrix}4\\4\end{pmatrix} = \begin{pmatrix}1.333\\1.333\end{pmatrix}$$

$$\mu_2 = \frac{1}{3}\left[\begin{pmatrix}8\\8\end{pmatrix} + \begin{pmatrix}9\\8\end{pmatrix} + \begin{pmatrix}8\\9\end{pmatrix}\right] = \frac{1}{3}\begin{pmatrix}25\\25\end{pmatrix} = \begin{pmatrix}8.333\\8.333\end{pmatrix}$$

Step 5: re-assign (iteration 2). For example, for $P_2 = (2, 1)$:

$$\begin{aligned}
d_1 &= \sqrt{(2-1.333)^2+(1-1.333)^2} = \sqrt{0.444+0.111} = \sqrt{0.555} \approx 0.745\\
d_2 &= \sqrt{(2-8.333)^2+(1-8.333)^2} = \sqrt{40.11+53.78} = \sqrt{93.89} \approx 9.690
\end{aligned}$$

P_2 stays in C_1. Checking all other points in the same way gives the *same* memberships: every point stays in its current cluster.

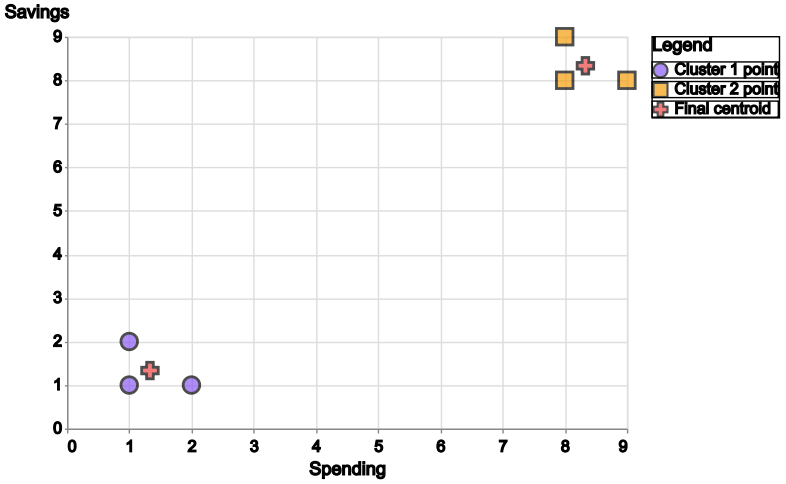
Step 6: stop. Recomputing the means gives exactly the same μ_1 and μ_2, so the centroids no longer change. The algorithm has **converged**.

Final value of the cost (the objective explained in the next section, using squared distances to the final centroids):

$$J_{C_1} = \underbrace{(\tfrac{1}{3})^2 + (\tfrac{1}{3})^2}_{P_1} + \underbrace{(\tfrac{2}{3})^2 + (\tfrac{1}{3})^2}_{P_2} + \underbrace{(\tfrac{1}{3})^2 + (\tfrac{2}{3})^2}_{P_3} = \frac{2+5+5}{9} = \frac{4}{3}$$

$$J_{C_2} = \frac{2+5+5}{9} = \frac{4}{3}$$

$$J = J_{C_1} + J_{C_2} = \frac{8}{3} \approx 2.667$$



---

13. What Is K-Means Actually Doing?

The procedure (random values, Euclidean distance, assign, update, reassign, repeat) is easy to memorise. A more interesting question is *what exactly is being achieved*.

Mathematically, **K-Means is an optimization problem.** It is minimizing an **objective (cost) function** J:

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

How to read this:

- The inner sum runs over all points x_i belonging to cluster C_k.
- $\|x_i - \mu_k\|^2$ is the **squared Euclidean distance** between a point and its cluster centroid.
- The outer sum runs over all K clusters.

So K-Means is **minimizing the Euclidean distance (squared) between each data point x_i and its cluster centre**, while the centre itself keeps being updated in between.

Link to something familiar. J has the same form as the **mean squared error** used in regression and classification. Even though we are doing clustering, the cost function looks very familiar.

Why do the two alternating steps both reduce J?

- *Assignment step:* for fixed centroids, giving each point to its nearest centroid can only lower (or keep) each point's squared distance.
- *Update step:* for a fixed cluster membership, the best centre of a cluster is its **mean**. To see this, set the derivative of the cluster's cost to zero:

$$\frac{\partial}{\partial \mu_k} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 = -2 \sum_{x_i \in C_k} (x_i - \mu_k) = 0 \implies \mu_k = \frac{1}{n_k} \sum_{x_i \in C_k} x_i$$

This matches the update formula in Step 3 exactly.

Because J never increases and cannot go below zero, the algorithm eventually stops, which is the "μ_k no longer changing" condition.

> **Note for Review Pro-Tip:** An exam may ask "what does K-Means minimize?". The answer is the **sum of squared Euclidean distances from each point to the centroid of its cluster** (the within-cluster sum of squares). Remember that *the mean is the minimizer of squared distance*, and that is exactly why the centroid update uses an average. The lecture only introduced the cost function and promised a deeper look next time.

14. Summary of the Lecture

Topic	Key Takeaway
Supervised vs unsupervised	Ask one question: is a label or target available?
Classification	Labels known, fixed target, supervised.
Clustering	No labels, more general, more interesting, more challenging, unsupervised.
Post-analysis	The algorithm gives clusters. **You** explain why they exist and what to do about them.
Domain knowledge	Essential for deciding between classification and clustering and for interpreting results.
Client requests	Clients state broad goals (such as maximize profit). The data engineer chooses the method and delivers actionable insights.
K-Means	Choose K, initialise centroids randomly, assign to nearest centroid, update centroids as means, repeat until μ_k stops changing.
K-Means objective	Minimizes $J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$, which has the same form as mean squared error.

The main message: it is not just *which tool you can apply* (K-Means, fuzzy clustering, PCA, DFT and so on). It is whether you understand from the data's perspective **which method fits** and whether you can **translate the client's demand into a clustering (or classification) problem**. Always build a physical intuition for *why* you are doing what you are doing. The next lecture continues with a deeper look at K-Means.

15. Practice Questions

Q1. The Missing Labels Dilemma. A hospital has 10 lakh patient records with blood pressure, cholesterol, age and a column "heart attack within 5 years (Yes/No)". A colleague proposes ignoring that column and running K-Means with $K = 4$ instead of building a classifier. Argue when this choice could be wiser than classification, and how you could still use the ignored column afterwards to judge whether the four clusters mean anything.

Q2. Same Data, Different Story. A bank's customer table has (spending, savings, investment). Clustering gives two clusters, but the bank manager says "so what?". Describe the secondary analysis you would carry out, what extra information from the bank you would request, and give two concrete actionable insights you could report. Why is stating them as "options to consider" more honest than "what you must do"?

Q3. One Iteration by Hand. Take the points $(1, 2),\ (2, 1),\ (6, 5),\ (7, 7)$ with $K = 2$ and initial centroids $\mu_1 = (1, 1)$ and $\mu_2 = (7, 7)$. Perform two full iterations (distances, assignment, centroid update). Does the algorithm converge in two iterations? Compute the cost J after each iteration and verify that it did not increase.

Q4. Why the Mean? Suppose that, instead of replacing each centroid by the **mean** of its cluster, someone replaces it by the **farthest point** of the cluster. Using the cost function J, explain why the algorithm may no longer be guaranteed to reduce J at every step. Then show, for a cluster with one-dimensional points $\{1, 2, 9\}$, which of the two choices (mean $= 4$ versus the point 9) gives the lower sum of squared distances, and compute both values.

Q5. When Intuition Meets Geometry. Look back at the picture of a ring of points surrounding an inner group, with a third group on the side. If you run K-Means with $K = 3$, where do you expect the centroids to end up, and why might the "ring" be split unnaturally? What does this tell you about checking K-Means results with domain knowledge and visualisation, instead of trusting the algorithm's output blindly?